Robust Training of Neural Networks at Arbitrary Precision and Sparsity

C. Ye, G. Chu, Y. Liu, Y. Zhang, L. Lew, L. Zhang, M. Sandler, A. Howard
Google DeepMind • ICLR 2026

Presented by

Ayush Srivastava (2026AIB2015) Sarvesh Nikas (2026AIB2045)

Why quantization, and the forward pass during training

Why quantization.

- Billions of weights, each 32 or 16 bits.
- Too big for a phone, slow, costly in energy.
- Quantization: 4, 2 or 1 bit per weight.

Training with quantization.

- Training keeps the float weights x.
- Forward pass uses quantized $\hat{x}$; backward pass updates the float x.
- Inference keeps only $\hat{x}$ (stored as q plus s_f, b_f per block).

The forward pass during training.

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{s_f q + b_f \mathbf{1}} \hat{x} \longrightarrow \text{rest of network} \longrightarrow L$$

- $x \in \mathbb{R}^N$: float block, kept and updated during training.
- $f(x) = \dfrac{x - b_f \mathbf{1}}{s_f}$: rescales x onto the k-bit integer grid $\{0, \ldots, 2^k - 1\}$, with $s_f = \dfrac{x_{\max} - x_{\min}}{2^k - 1}$, $b_f = x_{\min}$.
- $q = \text{round}\big(f(x)\big) = f(x) + \delta$: the integers, with rounding error $|\delta_i| \le \frac{1}{2}$.
- $\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta$: x as seen after quantization; this is what the next layer uses.

The problem: the true gradient is zero

Chain rule through the quantizer. Gradient descent needs $\nabla_x L$:

$$\nabla_x L = \Big(\frac{\partial \hat{x}}{\partial x}\Big)^\top \nabla_{\hat{x}} L, \qquad \frac{\partial \hat{x}}{\partial x} = \underbrace{s_f}_{\text{scale back}} \cdot \underbrace{\frac{\partial q}{\partial f(x)}}_{\text{round}} \cdot \underbrace{\frac{1}{s_f}}_{f} = \frac{\partial q}{\partial f(x)}.$$

The two scalings cancel: the local gradient is exactly the slope of round.

Round is a staircase.

$$\frac{d}{du}\,\mathrm{round}(u) = 0 \quad \text{for } u \notin \mathbb{Z} + \tfrac{1}{2}, \qquad \text{undefined at } u \in \mathbb{Z} + \tfrac{1}{2}.$$

Hence $\dfrac{\partial q}{\partial f(x)} = 0$ almost everywhere, so

$$\nabla_x L = 0.$$

Consequence. A small change in x does not change q, so the loss gives x no direction to move: x and every layer before it never update. Training cannot start.

The standard fix: Straight-Through Estimator (STE)

Idea. Round as before in the forward pass, but treat the rounding error as a constant in the backward pass:

$$q = f(x) + \delta, \qquad \delta = \text{stopgrad}\Big(\text{round}\big(f(x)\big) - f(x)\Big),$$

where stopgrad keeps the value in the forward pass and has zero derivative in the backward pass.

Forward vs. backward.

$$\text{forward:}\quad \hat{x} = x + s_f\,\delta, \qquad\qquad \text{backward:}\quad \frac{\partial \hat{x}}{\partial x} = s_f \cdot I \cdot \frac{1}{s_f} = I, \quad \nabla_x L = \nabla_{\hat{x}} L.$$

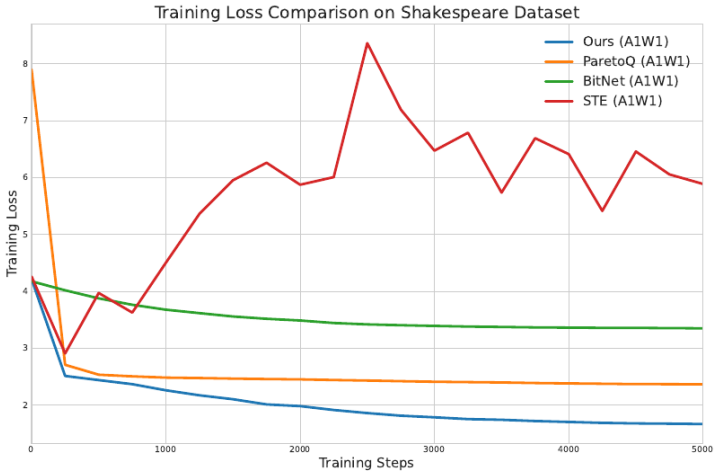
The gradient reaching x is no longer zero, so training works.

The blind spot.

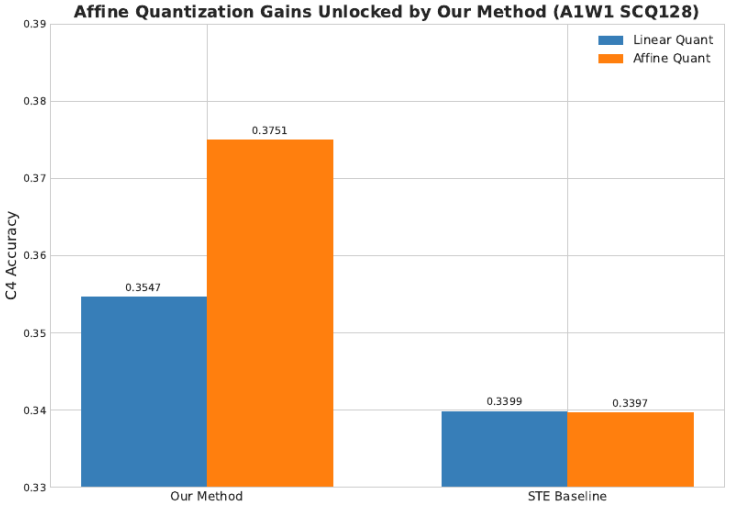
- The forward pass shifts every entry by $s_f\delta_i$, but the Jacobian is I, exactly as with no quantization. δ does not appear in it.
- So earlier layers get no signal about how rounding hurts the loss, and cannot learn to be robust to it.
- At 8 bits $s_f\delta$ is tiny and this is harmless. At 1–2 bits it is as large as the values themselves: STE diverges.

Reference: Y. Bengio, N. Léonard, A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv:1308.3432, 2013.

STE at 1 bit: results from the paper



Training loss, 1-bit weights and activations (A1W1), small GPT on Shakespeare. STE (red) diverges; the paper's method (blue) converges.



Accuracy at A1W1, linear vs. affine quantization. Under STE affine gives no gain; with the paper's method it does.

Source: Ye et al., "Robust Training of Neural Networks at Arbitrary Precision and Sparsity," ICLR 2026, Fig. 1.

The new approach: a denoising reconstruction g

The change. Keep f and round. Replace $\hat{x}$ by a fitted reconstruction g.

- **Reconstruction.** A new affine map, fitted to x:

$$\hat{x} = s_f\, q + b_f \mathbf{1} \quad \longrightarrow \quad g(q) = s_g\, q + b_g \mathbf{1}$$

- **Ridge regression objective.** Estimate s_g and b_g from the data:

$$\min_{s_g,\, b_g} \; \frac{1}{2N} \left\| s_g\, q + b_g \mathbf{1} - x \right\|^2 + \frac{\lambda}{2}\, s_g^2$$

- **Solution.** Ridge regression has a closed form:

$$s_g = \frac{\mathrm{Cov}_{xq}}{\mathrm{Var}_q + \lambda}, \qquad b_g = \bar{x} - s_g\, \bar{q}, \qquad \lambda = 0.01$$

λ *is the denoising knob.*

Forward pass. The next layer gets $g(q)$, not $\hat{x}$.

$$x \xrightarrow{\;f\;} f(x) \xrightarrow{\;\text{round}\;} q \xrightarrow{\;g\;} g(q) \longrightarrow \text{rest of network} \longrightarrow L$$

Backward pass. Round stays straight through, as in STE. But g now depends on δ.

$$\nabla_x L \xleftarrow[\text{straight through}]{\times\, 1/s_f} \nabla_q L \xleftarrow[\text{depends on } \delta]{\times\, (\partial g/\partial q)^\top} \nabla_g L \longleftarrow \text{rest of network} \longleftarrow L$$

Why it works: the Jacobian trick

The idea. Both methods guess round's gradient the same way. Only the Jacobian differs*:

$$J_{\mathrm{STE}} = \frac{\partial}{\partial x}\big[s_f\, q + b_f \mathbf{1}\big] = \underbrace{0}_{\text{exact}} + \underbrace{I}_{\text{guess}} \qquad J_g = \frac{\partial}{\partial x}\big[s_g(x,q)\, q + b_g(x,q)\, \mathbf{1}\big] = \underbrace{P}_{\text{exact}} + \underbrace{\rho\,(I-P)}_{\text{guess}} + \underbrace{c\,u\,r^\top}_{\text{error}}$$

The three parts of J_g.

- **Exact.** Moving x moves the fitted levels (the reconstruction levels):

$$P = \frac{\mathbf{1}\mathbf{1}^\top}{N} + \frac{u\,u^\top}{N(\mathrm{Var}_q + \lambda)}, \qquad u = q - \bar{q}\mathbf{1}$$

- **Guess.** Scaled by ρ: the fitted scale s_g over the grid scale s_f.

$$\rho = \frac{s_g}{s_f} = 0.29, \quad 0.87, \quad 0.99 \qquad \text{at 1, 2, 4 bits}$$

Coarse rounding gives a small ρ, so the guess counts less.

- **Error.** Carries the rounding error into the gradient:

$$r = x - g(q), \qquad c = \frac{1}{s_f N(\mathrm{Var}_q + \lambda)}$$

*Derived by us, not in the paper. Checked against autograd. ρ measured on a random block of 64 values.

Results from the paper

Small GPT on Shakespeare, 1-bit weights and activations. STE's loss blows up; g trains smoothly (slide 5).

Gemma 3 language model (1B), trained from scratch. Score: share of next tokens predicted correctly.

bits for weights and activations	STE	with g	gain
4	0.425	0.427	+0.002
2	0.379	0.403	+0.023
1	0.340	0.375	+0.035

The fewer the bits, the bigger the gain.

- **STE can diverge.** At 1.5 bits $(-1, 0, 1)$, one STE run ends in NaN. g still trains, reaching 0.330.

Bigger model, fewer bits. Quantized Gemma 3 4B beats 1B.

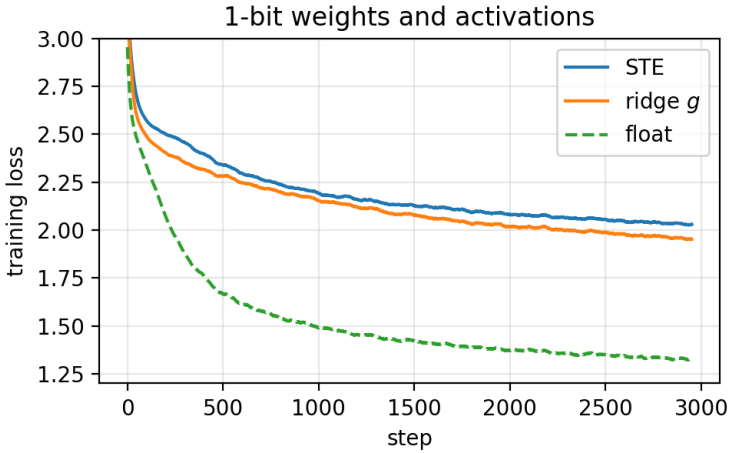
- 4B, 1-bit weights, half pruned: 650 MB, score 0.452.
- 1B, full precision: 1400 MB, score 0.449.

Other tasks, 4 bits, no extra tuning. Matches full precision.

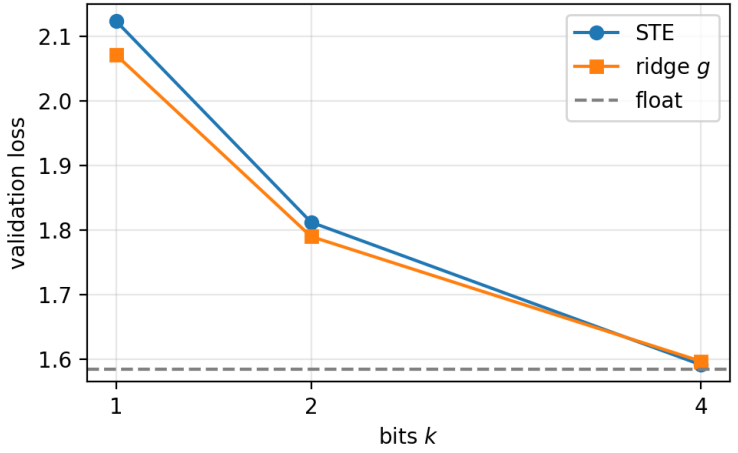
- Images (ResNet-50): 76.45% vs. 76.41%.
- Translation (English–German): 29.71 vs. 29.49 BLEU.

Toy implementation

Setup. Character-level GPT (4 blocks, 0.82M parameters) on Tiny Shakespeare. All linear layers in the blocks quantized to k bits (weights and activations, affine, per row); embeddings and output layer full precision. 3000 steps, same seed; only the dequantization differs: STE ($\hat{x}$) vs. ridge (g, $\lambda = 0.01$), $k = 1, 2, 4$.



Training loss at A1W1.



Final validation loss.

Gradient check (1 bit). Same q, δ changed by 0.106: the gradient reaching x changes by 0.0000 under STE and 0.0144 under ridge.
Size. Float32 model 3.19 MB; at 1 bit 250 KB (12.7× smaller).

Reference: model follows nanoGPT by A. Karpathy, `github.com/karpathy/nanoGPT`.

Advantages and disadvantages

Advantages.

- **Makes 1-bit training work.**

  1-bit matmul is just XNOR + bit-count,
  256× *cheaper than 16-bit.*

  STE can diverge there; g trains smoothly.

- **Better reconstruction.**

  Closer to x than standard dequantization.

- **Extends to sparsity.**

  Sparsity is quantization with small values thresholded to 0:

  $$x \xrightarrow{\text{threshold}} x_S \xrightarrow{f,\ \text{round}} q \xrightarrow{g} g(q)$$

Disadvantages.

- **Round is still a guess.**

  "Without gradient estimation" is too strong.

- **Small gain at 4 bits.**

  Only +0.002, *in the most common setting.*

- **Extra storage.**

  A scale and shift per 128 weights.

- **Savings are estimates.**

  No real hardware, no speed numbers.

Our takeaways

What convinced us.

A clean comparison.

Only the reconstruction differs between the two methods.

Consistent gains.

g wins in every setting, more at fewer bits.

Our toy agrees.

Better at 1–2 bits, a tie at 4 bits.

What we would question.

"No gradient estimation".

The paper claims it (abstract; Table 8).

But δ is detached, exactly as in STE:

$$q = f(x) + \text{stopgrad}(\delta) \;\Rightarrow\; \partial q / \partial f(x) = I$$

"For the first time".

LSQ (Esser et al., 2019) already puts δ in a gradient.

What we would try next. More seeds, a larger toy model, a sweep over λ.

Takeaway: Do not ignore the rounding error. Fit it.
A fitted reconstruction cleans the forward pass. It makes the backward pass error-aware.